

DEV-589: Analyze current Redis ACL configuration at Glynac

Department: DevOps

Prepared by: Thanusha Bai V

Date: 17-sept-2026

Task: DEV-587-Redis Security Hardening

1. Executive Summary

This report documents the ACL configuration audit of the Redis instance under analysis. Due to lack of direct access to Glynac's production Redis deployment, the audit was conducted on a **freshly installed reference instance** (Redis 8.10.1 on Ubuntu) representing the default, unhardened state. The findings establish the baseline that Glynac's actual deployments will be measured against. Key result: the default Redis configuration provides **no authentication, no user isolation, and full command access to any client** – a state that must be corrected before production use. A reusable audit script and fill-in template are included for future audits against Glynac's live environment.

2. Audit Scope & Methodology

Item	Detail
Target	Reference Redis instance (default install)
Redis version	8.10.1
Environment	Ubuntu VM, localhost-only
Access	Direct redis-cli
Commands used	ACL WHOAMI, ACL GETUSER, ACL LOG, ACL CAT, CONFIG GET
Limitation	Not Glynac's production instance – findings reflect default Redis behavior

3. Findings

3.1 Current Authenticated User

```
ACL WHOAMI → "default"
```

All connections authenticate as the default user by default – meaning **no credentials are required** to connect.

3.2 Default User Permissions

```
ACL GETUSER default
flags:      on, nopass, sanitize-payload
passwords:  (empty)
commands:   +@all
keys:       ~*
channels:   &*
```

Field	Value	Risk
Status	on	Enabled by default
Password	nopass / empty	No authentication
Commands	+%all	Every command allowed, including CONFIG, FLUSHALL, DEBUG, EVAL
Keys	~*	Full access to all keys
Channels	&*	Full access to all Pub/Sub channels

Verdict: The default user is effectively **unauthenticated root**. Anyone who can reach port 6379 has complete control.

3.3 Global Password Setting

```
CONFIG GET requirepass → ""
```

No global password is set. Authentication is entirely disabled.

3.4 Network Exposure

```
CONFIG GET protected-mode → "yes"
```

```
CONFIG GET bind → "127.0.0.1 -::1"
```

```
CONFIG GET port → "6379"
```

Setting	Value	Verdict
protected-mode	yes	✅ Good – blocks non-loopback connections when no password set
bind	127.0.0.1 -:::1	✅ Good – localhost only
port	6379	⚠️ Default port; firewall rules still required

Note: Network exposure is currently safe **only because of the localhost bind**. If bind were changed to 0.0.0.0, the instance would be immediately exploitable.

3.5 ACL Security Events

ACL LOG → (empty array)

No authentication failures or permission denials logged – expected, since no authentication is required and no ACL restrictions exist.

3.6 Command Categories (Redis 8.10.1)

Total categories: **30**

Notable categories:

- dangerous (93 commands including config|set, flushall, debug, replicaof, acl|setuser)
- admin, scripting, transaction, connection, pubsub
- Module categories: bloom, cuckoo, cms, topk, tdigest, search, json, timeseries

Key observation: The default user currently has +@all, meaning it can execute **every command in every category** – including all 93 dangerous commands.

4. ACL Audit Checklist (Glynac Template)

Use this when auditing Glynac's live Redis instances:

#	Check	Expected	Glynac Result	Status
1	Default user disabled (off)	off	–	☐
2	Default user has no password OR strong password	Strong	–	☐
3	Each service has its own ACL user	≥1 per service	–	☐
4	Service users use ~pattern (not ~*)	Scoped	–	☐
5	Dangerous commands denied (-@dangerous)	Denied	–	☐
6	CONFIG, FLUSHALL, DEBUG denied	Denied	–	☐
7	requirepass set as fallback	Set	–	☐
8	protected-mode yes	Yes	–	☐
9	bind restricted to private interfaces	Private	–	☐
10	ACL LOG	Reviewed	–	☐

	reviewed for denied commands			
11	ACL SAVE run after changes	Run	–	☐
12	ACL file persisted (aclfile configured)	Yes	–	☐

5. Reference Audit Script

Save as `redis-acl-audit.sh` – can be run against any Redis instance:

```
#!/bin/bash
# DEV-589: Redis ACL Audit Script
# Usage: ./redis-acl-audit.sh [host] [port]

HOST=${1:-127.0.0.1}
PORT=${2:-6379}
RC="redis-cli -h $HOST -p $PORT"

echo "=== Redis ACL Audit - $(date) ==="
echo "Target: $HOST:$PORT"
echo

echo "--- Version ---"
$RC INFO server | grep redis_version

echo "--- Current User ---"
$RC ACL WHOAMI

echo "--- Default User Permissions ---"
$RC ACL GETUSER default

echo "--- All ACL Users ---"
$RC ACL LIST

echo "--- Global Password Status ---"
$RC CONFIG GET requirepass

echo "--- Network Bindings ---"
$RC CONFIG GET bind
$RC CONFIG GET protected-mode
$RC CONFIG GET port

echo "--- Recent ACL Security Events ---"
$RC ACL LOG

echo "--- Dangerous Command Category ---"
$RC ACL CAT dangerous

echo "=== Audit Complete ==="
```

6. Summary of Findings (Reference Instance)

Area	Status	Notes
Authentication	 Disabled	nopass, empty requirepass
User isolation	 None	Only default user exists
Command restrictions	 None	+@all granted
Key restrictions	 None	~* granted
Channel restrictions	 None	&* granted
Network binding	 Safe	127.0.0.1 only
Protected mode	 Enabled	Default safety layer active
ACL logging	 Empty	No events to review

7. Recommendations for Glynac

1. **Disable the default user** (ACL SETUSER default off) on all instances.
2. **Create service-specific users** with scoped key patterns (detailed in DEV-590).
3. **Deny @dangerous commands** for all non-admin users.
4. **Set requirepass** as a fallback layer even when ACLs are used.
5. **Persist ACL changes** via aclfile config and ACL SAVE.
6. **Run the audit script** (Section 5) against every Glynac Redis instance once access is granted, and record

results in the checklist (Section 4).

7. **Review ACL LOG** periodically for denied-command attempts – indicator of misconfiguration or attack.
-

This analysis establishes the baseline for Glynac's Redis ACL hardening. Actual Glynac instance results are pending access provisioning.

END OF REPORT